

PROJECT DIRECTORY STRUCTURE

Medical Appointment No-Show Prediction System

Document Type	Project Directory / Repository Structure
Related Documents	Synopsis, PRD v1.0, TRD v1.0, FRD v1.0
Version	1.0 (Draft)
Owner	Engineering Team

1. Purpose

This document defines the recommended repository / folder structure for implementing the Medical Appointment No-Show Prediction System, consistent with the architecture and modules described in the TRD and FRD. It gives engineers a common layout for data, source code, models, tests, configuration, deployment, and documentation.

2. Directory Tree

```
medical-noshow-prediction/
|
+-- data/
|   +-- raw/                # Original, unmodified source data
|   +-- processed/          # Cleaned, feature-engineered datasets
|   +-- external/           # Reference/lookup data from external sources
|
+-- notebooks/
|   +-- 01_eda.ipynb        # Exploratory data analysis
|   +-- 02_feature_engineering.ipynb
|   +-- 03_model_experiments.ipynb
|
+-- src/
|   +-- data/
|   |   +-- ingestion.py    # Load data from source systems / files
|   |   +-- preprocessing.py # Cleaning, encoding, deduplication
|   |   +-- validation.py   # Schema and data-quality checks
|   |
|   +-- features/
|   |   +-- feature_engineering.py # waiting_days, age_group, prior_noshow_rate, etc.
|   |
|   +-- models/
|   |   +-- train.py        # Model training entry point
|   |   +-- evaluate.py     # Metrics: accuracy, precision, recall, F1, ROC-AUC
|   |   +-- registry.py     # Model versioning / registry interface
|   |   +-- config/
|   |       +-- model_params.yaml # Hyperparameters per algorithm
|   |
|   +-- api/
|   |   +-- main.py         # API app entry point
|   |   +-- routes/
|   |   |   +-- predict.py  # POST /api/v1/predict
|   |   |   +-- batch_predict.py # POST /api/v1/predict/batch
|   |   +-- schemas.py     # Request/response models
|   |   +-- auth.py        # Authentication / API key handling
|   |
|   +-- dashboard/
|   |   +-- app.py          # Dashboard entry point
|   |   +-- components/
|   |   |   +-- risk_list.py # Risk-ranked appointment list view
|   |   |   +-- analytics_charts.py
|   |   +-- assets/        # Static assets (CSS, icons)
|   |
|   +-- utils/
|   |   +-- logging.py
|   |   +-- db.py          # Database connection helpers
|
+-- models_registry/        # Versioned trained model artifacts (gitignored)
```

```

|   +-- v1.0/
|   +-- v1.1/
|
+-- tests/
|   +-- unit/
|   |   +-- test_preprocessing.py
|   |   +-- test_features.py
|   |   +-- test_api_schemas.py
|   +-- integration/
|   |   +-- test_pipeline_end_to_end.py
|   |   +-- test_scoring_api.py
|   +-- test_data/
|       +-- sample_appointments.csv
|
+-- config/
|   +-- config.yaml           # Environment-level settings
|   +-- logging.yaml
|   +-- thresholds.yaml      # Risk tier thresholds (Low/Medium/High)
|
+-- deployment/
|   +-- Dockerfile
|   +-- docker-compose.yml
|   +-- ci-cd/
|       +-- pipeline.yaml
|
+-- docs/
|   +-- Synopsis.pdf
|   +-- PRD.pdf
|   +-- TRD.pdf
|   +-- FRD.pdf
|   +-- Directory_Structure.pdf
|
+-- .env.example
+-- requirements.txt
+-- README.md
+-- .gitignore

```

3. Top-Level Folder Descriptions

Folder	Purpose
data/	Raw, processed, and external datasets used for training and testing (typically gitignored beyond small samples).
notebooks/	Exploratory analysis and experimentation notebooks; not used in production.
src/	All production source code, organized by responsibility: data, features, models, api, dashboard, utils.
models_registry/	Versioned, trained model artifacts and their metadata, as referenced in the TRD's Model Registry component.
tests/	Unit and integration tests, plus small sample datasets for testing.
config/	Environment configuration, logging setup, and business-rule thresholds (e.g., risk tiers).
deployment/	Containerization and CI/CD configuration for building and shipping the system.

Folder	Purpose
docs/	Project documentation, including the Synopsis, PRD, TRD, FRD, and this directory structure document.

4. Mapping to TRD Components

The `src/` subfolders correspond directly to the system components defined in the TRD's architecture (Section 2):

Source Folder	TRD Component
<code>src/data/</code>	Data Ingestion, Preprocessing
<code>src/features/</code>	Feature Engineering
<code>src/models/</code>	Model Training & Evaluation, Model Registry
<code>src/api/</code>	Scoring API
<code>src/dashboard/</code>	Dashboard / Staff Workflow

5. Naming Conventions

- Python files and folders: lowercase with underscores (snake_case), e.g., `feature_engineering.py`.
- Notebooks: numbered prefix indicating order of use, e.g., `01_eda.ipynb`.
- Model artifact folders: semantic version numbers, e.g., `v1.0/`, `v1.1/`.
- Config files: environment-specific suffixes when needed, e.g., `config.dev.yaml`, `config.prod.yaml`.
- Test files: prefixed with `test_` and mirroring the module they test.

6. Notes

- `data/` and `models_registry/` contents should be excluded from version control via `.gitignore`, except small fixtures needed for tests.
- `requirements.txt` (or a `pyproject.toml`/poetry setup) should pin versions for Pandas, NumPy, Scikit-learn, XGBoost, and API/dashboard frameworks.
- The `docs/` folder keeps all four project documents (Synopsis, PRD, TRD, FRD) alongside this structure document for single-source reference.